

(CSBB-254)

PROTRACK

(PROJECT MANAGEMENT PLATFORM)

ASSIGNMENT-01

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

(CSE-2, G-2)

Submitted By

VIVEK

(241210132)

&

PRANSHU SINGLA

(241210092)

2nd Year

Submitted To

Dr. SAHIL



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NATIONAL INSTITUTE OF TECHNOLOGY DELHI

Software Metrics Analysis

Metric 1: Cyclomatic Complexity

Cyclomatic Complexity (CC) is a code quality metric introduced by Thomas McCabe in 1976. It counts the number of independent paths that can be taken through a function or module. Every time the code makes a decision — like an if statement, a loop, or a switch case — it creates a new path. More paths mean more things that can go wrong and more test cases needed to cover the logic fully.

The formula is: $CC = E - N + 2P$ where E is the number of edges (transitions) in the control-flow graph, N is the number of nodes (statements), and P is the number of independent components (usually one per function).

A CC value between 1 and 5 means the function is simple and easy to test. A value between 6 and 10 is acceptable but worth reviewing. Anything above 10 is considered complex and increases the chance of bugs. Values above 20 are seen as unmaintainable and must be refactored immediately.

In ProTrack, several processes contain significant decision-making logic. The authentication flow involves checking whether the Google OAuth login succeeded, assigning the correct role (Student, Professor, or Admin), and handling token expiry. This gives it an estimated CC of around 7 to 9.

The class management module checks whether the professor role is valid, whether the class code already exists, and whether the enrolment limit has been reached — pushing its CC to roughly 6 to 8. The most complex module is the project submission validation, which checks submission completeness, deadline status, group membership, and duplicate detection all within one function. This results in an estimated CC of 10 to 14, which is above the acceptable range.

Authentication Flow (CC): Estimated 7 – 9

Class Management (CC): Estimated 6 – 8

Track Progress (CC): Estimated 8 – 10

Project Submission Validation (CC): Estimated 10 – 14 (above target)

Role-Based Middleware (CC): Estimated 9 – 12

Project target: $CC \leq 10$ for every function

The submission validation process is the highest-risk function and should be broken into smaller helper functions such as `checkDeadline()`, `checkGroupMembership()`, and `checkCompleteness()`. This will bring each function's CC back into the acceptable range and make individual testing much easier.

Metric 2: Defect Density

Defect Density is a reliability metric that tells us how many confirmed bugs exist per thousand lines of code (KLOC). It helps identify which parts of a system have the most problems and need more testing or refactoring. A lower defect density means the software is more stable and production-ready.

The formula is: **Defect Density = Total Defects / Size in KLOC**

Industry standards suggest that high-quality commercial software at release should have a defect density below 0.5 per KLOC. During integration testing, values between 1.0 and 3.0 per KLOC are typical. The overall development average across the industry is around 4.0 to 6.0, which reflects code that is still in active development and has not been thoroughly tested yet.

Based on the architectural complexity visible in the DFDs and the number of conditions in each module, the following defect estimates have been derived for ProTrack during the testing phase. The project management module has the most code at around 5.5 KLOC and carries an estimated 12 bugs, giving it a defect density of 2.18. The evaluation module, despite being smaller at 3.0 KLOC, has 7 estimated defects — pushing its density to 2.33, making it the highest-risk module.

User Auth & Profile (3.2 KLOC, 6 bugs): Density: 1.88 defects/KLOC

Class Management (2.8 KLOC, 4 bugs): Density: 1.43 defects/KLOC

Project Management (5.5 KLOC, 12 bugs): Density: 2.18 defects/KLOC

Group Formation (2.2 KLOC, 5 bugs): Density: 2.27 defects/KLOC

Project Evaluation (3.0 KLOC, 7 bugs): Density: 2.33 defects/KLOC (highest)

Middleware & API Routes (2.5 KLOC, 3 bugs): Density: 1.20 defects/KLOC

Overall Average (19.2 KLOC, 37 bugs): Density: 1.93 defects/KLOC

Release Target: Below 1.0 defects/KLOC

The current overall density of 1.93 is acceptable for the testing phase but needs to come down before the system goes live. The evaluation and group formation modules should be prioritised for additional unit tests. Using Zod validation on both the frontend and backend will also catch many input-related bugs early.

Metric 3: Response Time

Response time measures how long the system takes to reply after a user performs an action. It is one of the most visible quality factors because slow responses directly affect how people feel about using the software. In an academic context like ProTrack, hundreds of students might try to submit projects near a deadline at the same time, which makes performance planning especially important.

The total response time is calculated as: **TRT = Network Latency + Server Processing Time + Database Query Time + Rendering Time**

For a web application like ProTrack, the general industry benchmark for a good user experience is a page becoming interactive within 2.5 seconds and API calls completing within 300 milliseconds. The Time to First Byte (TTFB) should ideally be under 600 ms for server-side rendered pages.

ProTrack's architecture introduces several response time factors. The MongoDB Atlas cloud database adds network latency to every query since it is hosted remotely. The NextAuth.js middleware validates a JWT token

on every authenticated request, adding a small but constant overhead. Server-side rendering (SSR) through Next.js also adds processing time before the page reaches the user.

Based on the sequence diagram, the five major user flows have been analysed for their expected response times. The login process involves a Google OAuth round-trip plus JWT issuance, so the target is under 1.5 seconds. The class join operation involves a database query and enrolment update — targeted at under 800 ms. The most demanding operation is the final project submission, which involves writing to the submission database and running multiple validation checks, and should complete within 1.2 seconds.

Google Login (OAuth + JWT): Target: under 1.5 seconds

Join Class (query + enrolment update): Target: under 800 ms

Create Project (validation + DB insert): Target: under 900 ms

Update Progress (two DB writes): Target: under 600 ms

Final Project Submission (validation + DB write): Target: under 1.2 seconds

Individual API Response (p95): Target: under 300 ms

Individual Database Query: Target: under 100 ms

To meet these targets, the team should add MongoDB indexes on frequently searched fields like studentID and classID. Enabling SWR data caching on the frontend (already in the architecture) will prevent repeated API calls for the same data. Monitoring tools like Vercel Analytics should be set up so the team is alerted if response times start rising above acceptable limits.

Metric 4: Coupling and Cohesion

Coupling and cohesion are two design quality metrics that measure how well a system is structured. Coupling refers to how much one module depends on another. When modules are tightly coupled, a change in one module often forces changes in others, making the system brittle and difficult to maintain. The goal is to keep coupling as low as possible.

Cohesion, on the other hand, measures how focused a module is on doing a single, well-defined job. A highly cohesive module only deals with one concern, which makes it easier to understand, test, and update without causing side effects elsewhere. The design principle of low coupling combined with high cohesion is considered a fundamental goal in good software engineering.

Coupling is measured using the Instability Index: $I = C_e / (C_a + C_e)$ where C_e is the number of modules this module depends on (outgoing), and C_a is the number of modules that depend on this one (incoming). A value of 0 means the module is very stable; a value of 1 means it is unstable and likely to break if anything it depends on changes.

ProTrack's service layer (AuthService, ClassService, ProjectService, GroupService, and EvaluationService) provides a clear structure for managing coupling. The authentication service is used by all five API route groups, giving it a high afferent coupling ($C_a = 5$) but only two outgoing dependencies (User_DB and NextAuth). This results in a low instability index of 0.29, which is ideal for a core service.

The middleware layer is the most stable part of the system with an instability index of just 0.14, since almost every API route depends on it but it only depends on NextAuth for session data. The ProjectService falls in the middle with an index of 0.50, which is acceptable but should be monitored as the codebase grows.

The EvaluationService is the most problematic, with an instability index of 0.75. It depends on three separate databases — Project_DB, Group_DB, and User_DB — which means any change to any of those models could affect the evaluation logic. Additionally, the cohesion of this service is weakened because it mixes data-fetching responsibilities with business logic for grading.

AuthService Instability Index: 0.29 — Stable (ideal for core auth)

Middleware Layer Instability Index: 0.14 — Very stable

ClassService Instability Index: Balanced, well-focused

ProjectService Instability Index: 0.50 — Acceptable, monitor growth

EvaluationService Instability Index: 0.75 — Too high, needs refactoring

Target for all services: Instability Index below 0.60

The most important fix is to split the EvaluationService into a read-only query part and a write/logic part. This will lower its Ce and bring the instability down. Also, no business logic should be written directly inside API route files — all logic must go through the service layer. This keeps the HTTP handling and domain logic cleanly separated.

Metric 5: Test Coverage

Test coverage measures what percentage of the source code is actually executed during automated testing. It answers the question: if a bug exists in a particular line or branch, would our tests catch it? High test coverage reduces the risk of hidden bugs making it to production. For a multi-role system like ProTrack, this is especially important — a bug in the role-based access control could allow a student to view or modify another group's project, which would be a serious academic integrity issue.

There are different levels of coverage. Statement coverage checks whether each line of code was run at least once. Branch coverage checks whether both the true and false outcomes of every condition were tested. Function coverage checks whether every function was called. Of these, branch coverage is the most important for ProTrack because the access control logic is entirely based on conditional checks.

The formula is: **Coverage % = (Lines or Branches Tested / Total Lines or Branches) × 100**

Currently, ProTrack does not have a formalised automated test suite. This is a significant risk, particularly for the evaluation module and the role-based middleware. The following coverage targets have been set based on the risk profile of each module.

The authentication module should reach at least 90% coverage because an untested auth bug could result in unauthorised access or broken session handling. The project management module also needs at least 90% coverage — it contains six sub-processes as shown in the Level-2 DFD, and any one of them failing silently could result in lost submissions or incorrect progress tracking.

The evaluation module requires the highest coverage at 95% or above because grade data errors are very difficult to recover from and have direct academic consequences. The middleware layer, which handles role-based access control for every single route in the system, must achieve 100% branch coverage without exception.

Authentication Module Target: 90% or above
Class Management Module Target: 85% or above
Project Management Module Target: 90% or above
Group Formation Module Target: 85% or above
Evaluation Module Target: 95% or above (highest priority)
Middleware / RBAC Layer Target: 100% branch coverage (mandatory)
Overall Project Target: Minimum 75% overall, enforced in CI/CD

To implement this, the team should adopt Jest for unit and integration testing of all service-layer functions and API routes. Playwright or Cypress should be used for end-to-end tests covering the five main user journeys: login, join class, create project, update progress, and submit project. Coverage reports should be generated using Istanbul (nyc) and integrated into GitHub Actions so that every pull request automatically shows whether coverage has gone up or down.